

DBMS VIVA PREPARATION

Airline Reservation System — 30 Questions & Answers

From Fundamentals → Design Choices → Hypotheticals

 Analogy	 Answer	 Why asked	Q## Question
---	--	---	--------------

SECTION 1 — DBMS FUNDAMENTALS

[Fundamentals]	
Q01	What is a Primary Key, and why did every table in your schema have one?
	Analogy: Think of a Primary Key like an Aadhaar number. Two people can share the same name or phone number, but no two people can have the same Aadhaar. It uniquely identifies every single row in a table with no exceptions.
	Answer: A Primary Key (PK) is a column or set of columns whose values uniquely identify every row in a table. It enforces entity integrity — no two rows can be identical, and the PK column can never be NULL. In our schema, every table uses a surrogate INT PK with AUTO_INCREMENT (e.g. passenger_id, booking_id) so that the system generates the unique identifier automatically without relying on business data that could change.
	Why the examiner asks this: Tests whether you understand the most fundamental constraint in relational databases. If you cannot explain PKs confidently, everything else collapses.
[Fundamentals]	
Q02	What is a Foreign Key, and give one specific example from your schema where it matters most?
	Analogy: A Foreign Key is like a reference number on a bank receipt. The receipt has an Account Number that must actually exist in the bank's records. If the account doesn't exist, the transaction is rejected. The FK does the same — it refuses to let you point to a row that doesn't exist.
	Answer: A Foreign Key is a column in one table that references the Primary Key of another table, enforcing referential integrity. In our schema, the most critical FK is booking.pnr_id → pnr_group(pnr_id) with ON DELETE CASCADE. This means every booking must belong to a valid PNR group, and if that PNR is deleted, all its booking records are automatically cleaned up. This prevents orphaned bookings floating in the database without a parent order.
	Why the examiner asks this: Examiners want to see that you can point to a specific real example and explain the consequence — not just define the term.
[Fundamentals]	
Q03	What is the difference between a UNIQUE constraint and a Primary Key?
	Analogy: In a college, the Roll Number is the Primary Key — every student must have one, and it cannot be blank. The email address has a UNIQUE constraint — no two students can share it, but a student could theoretically not have an email (NULL). One is mandatory identity, the other is just a uniqueness rule.



Answer: A Primary Key is a special UNIQUE constraint that additionally enforces NOT NULL. A table can have only one PK, but multiple UNIQUE constraints. In our schema, passenger.passport_number is UNIQUE (not the PK) because a passenger is identified internally by passenger_id (the PK), but their passport must still be globally unique. Similarly, passenger.email has a UNIQUE constraint — one email per account — but it is not how other tables join to passenger.



Why the examiner asks this: *A classic gotcha. Many students think UNIQUE and PK are the same thing. This shows you understand the subtle distinction.*

Q04

[Fundamentals]

What are the three types of relationships in a relational database, and where does each appear in your schema?



Analogy: *One-to-One: One person has one passport. One-to-Many: One teacher teaches many students. Many-to-Many: Many students enroll in many courses — you need a separate enrollment table to handle this.*



Answer: 1) One-to-One (1:1): booking → checkin. One booking segment has exactly one check-in record. Enforced by the UNIQUE constraint on checkin.booking_id. 2) One-to-Many (1:M): pnr_group → booking. One PNR can contain multiple passenger-flight segments. flight → delay_record is also 1:M — one flight can experience multiple sequential delay events. 3) Many-to-Many (M:N): flight ↔ crew, resolved into crew_assignment junction table. A crew member works multiple flights; a flight needs multiple crew members.



Why the examiner asks this: *The screenshot explicitly mentions this. The examiner wants you to name real tables, not just give textbook definitions.*

Q05

[Fundamentals]

What is a NULL value in SQL, and why did you allow NULLs in some columns but not others?



Analogy: *NULL is like a blank field on a form — not zero, not empty string, just unknown or not applicable. 'Number of children: 0' is different from leaving that field blank entirely. In your database, a crew member's duty_hours_logged is NULL until the flight lands — the value exists eventually, it's just not known yet.*



Answer: NULL means the value is absent, unknown, or not yet applicable — it is not zero or an empty string. We used NULL selectively: columns like booking.external_booking_ref are NULL for direct bookings (there is no OTA reference), and delay_record.actual_departure is NULL until the plane actually pushes back. Critical identity and financial columns (passenger.email, payment.amount, booking.flight_id) are NOT NULL because a booking without a flight or a payment without an amount is logically invalid.



Why the examiner asks this: *Demonstrates that you made conscious decisions about data completeness rather than just leaving NULLs everywhere.*

SECTION 2 — NORMALISATION

Q06

[Normalisation]

What is BCNF and why did you target it specifically instead of stopping at 3NF?



Analogy: Imagine a spreadsheet where one column fully determines another — like if knowing the PIN code tells you the city name. If that relationship sneaks into a table where PIN code is not the primary key, you have a BCNF violation. Every fact should be stored exactly once and determined only by the table's own primary key.



Answer: BCNF (Boyce-Codd Normal Form) states that for every non-trivial functional dependency $X \rightarrow Y$ in a table, X must be a superkey. It is stricter than 3NF, which allows some transitive dependencies to remain if they involve candidate keys. We chose BCNF because the schema must serve production airline operations where update anomalies are costly — changing a country name should not require updating thousands of city rows. 3NF would have tolerated the country_name dependency inside the city table; BCNF forced us to extract country as its own table.



Why the examiner asks this: This is the theoretical backbone of the entire schema. The examiner wants to know you understand WHY you chose BCNF, not just that you did.

[Normalisation]

Q07

Explain the transitive dependency you fixed in the geographic hierarchy and how you resolved it.



Analogy: Originally, if the city table stored country_name directly, then: $city_id \rightarrow country_code \rightarrow country_name$. This is a chain — $city_id$ does not directly determine $country_name$, it goes through $country_code$ first. That indirect chain is the transitive dependency. Fixing it means breaking the chain by giving country its own table.



Answer: The original un-normalised design had city containing country_code and country_name. This created: $city_id \rightarrow country_code$ (direct) and $country_code \rightarrow country_name$ (transitive). So $city_id \rightarrow country_name$ was transitive, not direct — violating BCNF. We resolved this by creating a standalone country table where country_code is the PK and country_name is a dependent attribute. City now only holds country_code as a FK. Changing 'India' to 'Republic of India' requires updating exactly one row in country, not every Indian city row.



Why the examiner asks this: This is the single most concrete BCNF example in the schema. If you can explain this clearly, you demonstrate real understanding of normalisation.

[Normalisation]

Q08

You intentionally violated BCNF in two places. Which ones, and why was the violation justified?



Analogy: Sometimes following rules perfectly creates new problems. A hospital stores a patient's blood type on both their profile and their emergency card — technically redundant, but if the profile system crashes during an emergency, the card saves their life. Controlled redundancy for safety is sometimes the right call.



Answer: Two intentional denormalisations: 1) baggage.excess_fee — this is calculated from weight vs fare_class.baggage_allowance_kg. We snapshot it at check-in time because if the airline changes its baggage policy next month, historical excess fee charges must remain legally accurate. 2) pnr_group.contact_email and contact_phone — the passenger may update their profile later, but we capture what was live at transaction time for receipt archival and dispute resolution. Both cases prioritise financial and legal auditability over strict normalisation.



Why the examiner asks this: Shows the examiner you understand that real-world design requires pragmatic trade-offs, not blind rule-following.

[Normalisation]

Q09

Why did you split aircraft_model and aircraft into two separate tables instead of one?



Analogy: Think of aircraft_model like a car model specification — a 'Toyota Camry 2024' spec defines engine size, seating, fuel type. Then each actual car on the road is a separate instance with its own registration plate. You would not copy-paste the full car specification onto every registration document — you just reference the model.



Answer: If aircraft stored model attributes (manufacturer, total_seat_capacity, seat_layout) directly, those values would be duplicated across every physical hull of the same type. All Boeing 737 MAX 8 aircraft would repeat 'Boeing', '189 seats', same layout. A BCNF violation: model attributes depend on model_id, not tail_number. Splitting into aircraft_model (blueprint) and aircraft (instance) means the specification is stored once. Updating seat capacity for a model type updates exactly one row — not every aircraft registered to that type.



Why the examiner asks this: Classic 'why two tables instead of one' design question. The examiner is testing whether you can articulate the redundancy problem.

[Normalisation]

Q10

booking_status is technically not in BCNF — you said so yourself. Defend this choice.



Analogy: A judge's case diary is not normalised — it repeats the case number on every entry. But that's the point: you need an immutable chronological log of every hearing, verdict, and appeal. You would never 'overwrite' a previous entry. The diary is not a relational table — it's an audit ledger.



Answer: booking_status stores multiple rows per booking_id with different status values and timestamps — this means booking_id is not a superkey, violating BCNF. However, it is intentionally an append-only audit ledger, not a traditional entity table. The correct analogy is event sourcing. We cannot overwrite 'confirmed' with 'cancelled' on the booking row because aviation regulations require a complete, tamper-evident history of all state changes. The ledger supports time-travel queries, dispute resolution, and regulatory compliance. The design deviation is deliberate, documented, and justified.



Why the examiner asks this: Tests intellectual honesty. Examiners respect students who acknowledge trade-offs and defend them, rather than claiming perfection.

SECTION 3 — DESIGN CHOICES

[Design Choice]

Q11

Why did you create a pnr_group table? Why not just put all booking fields directly into the booking table?



Analogy: When a family of four books a holiday package, they get one invoice and one confirmation number — but four airline tickets. The invoice (pnr_group) is the parent container. Each individual ticket (booking) is a line item. Merging them would mean repeating the contact email and payment details four times — once per passenger — which creates update anomalies.



Answer: pnr_group is the order envelope that aggregates multi-passenger, multi-leg itineraries under one PNR reference code. Without it, contact_email, contact_phone, and payment records would need to be duplicated on every individual booking row. A family of four on a two-leg trip would duplicate that data eight times. pnr_group also enables the correct payment model: billing is aggregated at the order level, not per passenger per leg. The structure mirrors IATA NDC Level 4's 'Order / Order Item' hierarchy — the same model used by Amadeus and Sabre.



Why the examiner asks this: *The examiner will probe whether you understand WHY this table exists. 'Because groups exist' is a weak answer. The payment and contact snapshot justification is the strong one.*

Q12

[Design Choice]

Why is the seat table scoped to flight_id and not to aircraft_id?



Analogy: *Think of a cinema multiplex. Screen 3 has 200 seats. On Monday's 3pm show, seat B12 is available. On Monday's 7pm show, seat B12 is booked. The seat itself hasn't changed — its availability is tied to the specific screening (flight instance), not the physical screen (aircraft).*



Answer: If seats were tied to aircraft_id, we could not independently track availability per flight instance. Seat 14A on aircraft tail number VT-TNA would appear 'booked' even for tomorrow's flight if today's booking was stored against the aircraft. Tying seats to flight_id allows the same physical seat to be 'available' on tomorrow's flight while 'booked' on today's. The session-level attributes — availability_status and hold_expires_at — belong at the flight-seat intersection, not on the aircraft blueprint. This is also how Amadeus's seat map inventory model works.



Why the examiner asks this: *Directly tests understanding of static vs dynamic data. A common beginner mistake is to attach availability to the aircraft.*

Q13

[Design Choice]

Why does the booking table have both seat_id and fare_class_id, when the seat already has a fare_class_id?



Analogy: *When you buy a train ticket at a discounted fare, your ticket records the fare class you paid for — even if the coach assignment changes later. The seat the train assigns you to might have a different label. The ticket is your proof of what you paid; the seat is just where you sit.*



Answer: The fare_class_id on seat defines which cabin class the physical seat belongs to. The fare_class_id on booking is a snapshot of the class at the time of booking. These can diverge: an airline may upgrade a passenger to business class (seat changes cabin), or apply a promotional economy fare that doesn't match the physical seat's default class. Storing fare_class_id on booking independently protects the financial record — what the passenger actually paid for — from being affected by operational seat reassignments.



Why the examiner asks this: *This is a 'why did you duplicate this field' gotcha. The answer turns on the snapshot concept.*

Q14

[Design Choice]

Explain the self-referential (unary) relationship on the booking table.



Analogy: *When a court case gets appealed, the new case references the original case number. The appeal is a new case — with its own record — but it carries a link back to the parent. The relationship is within the same 'cases' table.*



Answer: booking.rescheduled_from_id is a FK that references booking(booking_id) — the same table. When a passenger reschedules, we do not overwrite the original booking record. Instead, we insert a new booking row and set its rescheduled_from_id to point at the booking it replaced. This preserves the full reschedule chain — if a booking is rescheduled twice (A → B → C), each link is independently preserved. It is NULLABLE because original bookings have no prior reference. It is placed at the booking (segment) level so that rescheduling one leg of a multi-leg trip does not disturb the other legs.



Why the examiner asks this: *Unary relationships are unusual enough that the examiner will definitely ask about them. Knowing the chain-preservation argument is the key.*

Q15

[Design Choice]

Why is delay_record a 1:M child of flight instead of a 1:1 extension?



Analogy: *A flight's delay log is like a patient's fever chart. A patient doesn't get one temperature reading per stay — they get readings every few hours. Each reading is a separate event with its own timestamp and reason. You wouldn't overwrite the 10am reading when the 2pm reading comes in.*



Answer: A single flight can accumulate multiple delay events in sequence — an initial ATC hold, followed by a weather extension, followed by a maintenance check. If delay_record were 1:1, only the most recent or most severe delay could be stored, and the full operational narrative is lost. By making it 1:M, each discrete delay event gets its own row with its own delay_reason_code and delay_minutes. This is critical for operations reporting — the dashboard query for 'flights delayed beyond 2 hours' can correctly sum or compare individual delay events per flight.



Why the examiner asks this: *A specific assumption call-out in the schema doc. Shows the examiner you thought through real-world operational scenarios, not just the happy path.*

Q16

[Design Choice]

Why does crew_assignment exist as a separate table? Why not add crew_id directly to the flight table?



Analogy: *A flight needs a pilot, co-pilot, and four cabin crew. If you put crew_id on the flight table, you can only store one crew member per flight. You would need six columns (crew_id_1, crew_id_2... crew_id_6) — which breaks normalisation and falls apart when a flight needs a seventh crew member.*



Answer: Flight and crew have a Many-to-Many relationship — one crew member works multiple flights, one flight requires multiple crew members. Adding a crew_id column to flight would only support one crew member per flight. Instead, crew_assignment is a junction table that resolves M:N into two 1:M relationships. It also carries junction-level attributes: assigned_role (a pilot may act as co-pilot on a specific segment) and duty_hours_logged (belongs to the specific assignment, not globally to the crew member). UNIQUE(flight_id, crew_id) prevents the same crew member being double-assigned to one flight.



Why the examiner asks this: *The M:N resolution pattern is explicitly listed in the WhatsApp screenshot as a topic. You need to explain this with both the problem and the junction table solution.*

Q17

[Design Choice]

If a new airline — say IndiGo — wanted to use this system tomorrow, is your schema adaptable? What would need to change?



Analogy: *Think of the schema as a restaurant kitchen. It's designed to cook food — not specifically biryani for one customer. Adding a new customer (airline) is like adding a new order, not rebuilding the kitchen. But if the new customer needs a different cooking style (e.g. codeshare flights between two carriers), you might need a new station.*



Answer: The schema is largely carrier-agnostic. flight, route, aircraft, passenger, booking — none of these have an airline_id field. To make it explicitly multi-tenant, we would add an airline table (airline_id, airline_name, iata_carrier_code) and add airline_id as a FK on flight, route, and aircraft. The crew, fare_class, and loyalty_tier tables would also need airline scoping for isolation. The booking.source_channel CHECK constraint ('vistara_web', 'vistara_app'...) would need to be replaced with a dynamic lookup table to avoid hardcoded carrier names. The core relational structure requires no restructuring — just new FK columns and a new master table.



Why the examiner asks this: *Directly mirrors the 'switch company or add company' hypothetical from the WhatsApp screenshot. The examiner is testing schema extensibility thinking.*

SECTION 4 — INDEXING

[Indexing]

Q18

What is a database index and why do we need one? Does it have any downsides?



Analogy: *An index in a database works exactly like the index at the back of a textbook. Instead of reading every page to find 'normalisation', you jump straight to page 47. But adding that index takes extra pages in the book, and every time the content changes, the index must be reprinted too.*



Answer: An index is an auxiliary data structure (typically a B-Tree) that stores a sorted copy of one or more columns alongside pointers to the full rows. This allows the database engine to jump directly to matching rows instead of scanning the entire table. The B-Tree structure supports $O(\log n)$ lookups for equality and range queries. Downsides: each index consumes additional disk space; on every INSERT, UPDATE, or DELETE, all relevant indexes must also be updated, adding write overhead. This is why we only indexed columns that are frequently queried — not every column.



Why the examiner asks this: *The examiner will ask why you didn't just index every column. The write-overhead trade-off is the core answer.*

[Indexing]

Q19

Why did you NOT use a hash index on pnr_group(pnr_code), even though PNR lookup is an exact-match query where hashing would be faster?



Analogy: *A hash function is like a locker system where each item gets a fixed locker number. Finding one specific item is instant. But the lockers are randomly assigned — if you want 'all lockers from 100 to 200', you have to check all of them. A hash has no concept of order, so range queries and sorting are impossible.*



Answer: MySQL InnoDB does not support explicit hash indexes on disk. The CREATE INDEX ... USING HASH syntax is accepted but silently ignored — InnoDB creates a B-Tree index regardless. We chose to be explicit and create a B-Tree UNIQUE index on pnr_code. InnoDB does build an Adaptive Hash Index (AHI) in memory on hot B-Tree pages — so frequently accessed PNR codes do get near- $O(1)$ performance automatically, without us manually specifying HASH. Additionally, a hash index cannot enforce UNIQUE constraints — it was functionally incorrect to specify HASH + UNIQUE together in the original design.



Why the examiner asks this: *This is the gotcha from the WhatsApp screenshot. The examiner expects you to defend using B-Tree. Knowing about InnoDB's Adaptive Hash Index is what separates a strong answer from an average one.*

[Indexing]

Q20

Explain the composite index `idx_flight_route_date`. Why is column order important?



Analogy: Think of a physical phone directory sorted first by city, then by last name, then by first name. You can instantly find 'all numbers in Bangalore' or 'all Sharmas in Bangalore'. But if you want 'all Sharans across all cities', the directory is no help — the city column must be matched first before the name columns are useful.



Answer: `idx_flight_route_date` is defined as `(route_id, scheduled_departure)`. The left-prefix rule of B-Tree indexes means the index is only useful if the query's WHERE clause starts from the leftmost column. `Route_id` comes first because: 1) Flight search always specifies an origin-destination pair, making `route_id` an equality filter (high selectivity). 2) After filtering by route, the engine can range-scan `scheduled_departure` within that corridor for a specific date. Reversing the order — `(scheduled_departure, route_id)` — would force the engine to scan all flights on a date before filtering by route, which is far less selective.



Why the examiner asks this: Column order in composite indexes is a classic advanced interview question. Shows you understand how B-Trees work internally.

[Indexing]

Q21

What is a covering index? Which index in your schema achieves this effect?



Analogy: A covering index is like a restaurant menu that includes the price, calories, and allergens for every dish. You get all the information you need from the menu — you never need to call the kitchen. A non-covering index is a menu that just lists dish names — you still need to call the kitchen (the actual data row) for everything else.



Answer: A covering index is one where all columns needed by a query are present in the index itself, so the database engine can answer the query entirely from the index B-Tree without fetching the actual table row. In our schema, `idx_seat_inventory_lookup` is defined on `(flight_id, availability_status, fare_class_id, seat_number)`. The frequent query 'available seats by fare class for a selected flight' needs exactly these four columns. MySQL InnoDB does not support INCLUDE columns, so all four are added to the index key directly. The query resolves without a heap fetch, dramatically reducing I/O under high checkout traffic.



Why the examiner asks this: Covering indexes are a performance optimisation that distinguishes intermediate from advanced DB knowledge.

[Indexing]

Q22

Why did you not create a partial/filtered index for `boarding_status = 'checked_in'` in MySQL, even though that would be more efficient?



Analogy: In a library, a partial index would be like having a special catalog that only lists books currently on loan — a much smaller, faster list. MySQL doesn't support this kind of catalog. So instead you keep the full catalog and just tell the librarian 'only show me books marked as on loan' every time they search.



Answer: MySQL 8.0 does not support partial (filtered) indexes — indexes with a WHERE clause on a subset of rows. PostgreSQL does, which would allow: CREATE INDEX ... ON checkin (booking_id) WHERE boarding_status = 'checked_in'. In MySQL, we must create a full B-Tree index on (boarding_status, booking_id) covering all rows. We compensate by placing boarding_status as the leftmost index column so the engine can still do a selective prefix scan when the application queries WHERE boarding_status = 'checked_in'. The selectivity benefit is real — most passengers are not in the 'checked_in' state — but it comes from the query filter, not a filtered index.



Why the examiner asks this: Shows awareness of MySQL-specific limitations. The examiner may try to catch you claiming a MySQL feature that only exists in PostgreSQL.

SECTION 5 — JOINS & FREQUENT QUERIES

[Query Design]

Q23

Which tables would you join to answer: 'Find all flights between Mumbai and Delhi on 15 July 2025'?



Analogy: You are at a travel agency. To answer this, the agent checks: 1) the airport directory to find the Mumbai and Delhi airport IDs, 2) the route book to find corridors between those two airports, 3) the flight schedule to find which flights run on that route on that specific date.



Answer: You need: airport (to resolve 'Mumbai' → airport_id for BOM, 'Delhi' → airport_id for DEL) → route (to find route_id where origin_airport_id = BOM and destination_airport_id = DEL) → flight (to find flights on that route_id WHERE DATE(scheduled_departure) = '2025-07-15'). The index idx_flight_route_date on (route_id, scheduled_departure) directly serves this query. The join chain is: airport → route → flight. You do not need passenger or booking for this query.



Why the examiner asks this: This is the first frequent query listed in the problem statement. Examiners love asking you to walk through a real join chain.

[Query Design]

Q24

Which tables would you join to answer: 'Which passengers have checked in but not yet boarded for flight AI202?'



Analogy: The gate agent's screen shows all passengers whose boarding pass has been issued (checked in) but whose boarding pass hasn't been scanned at the gate yet (not boarded). They need to cross-reference the check-in system with the gate scan records.



Answer: Join: flight (filter WHERE flight_number = 'AI202') → booking (on flight_id) → passenger (on passenger_id) → checkin (on booking_id, WHERE boarding_status = 'checked_in'). The WHERE boarding_status = 'checked_in' condition is the key filter. The index idx_checkin_boarding_alert on (boarding_status, booking_id) supports this. You are looking for rows in checkin where boarding_time IS NULL or boarding_status = 'checked_in' — meaning the check-in record exists but the gate scan (boarding_time) has not been recorded.



Why the examiner asks this: Tests the ability to trace through the booking → checkin relationship and identify the correct filter condition.

[Query Design]

Q25

How would you calculate total revenue generated by route BOM→DEL for July 2025?



Analogy: To calculate revenue per route, think of it like tallying all sales receipts (payments) for orders (PNRs) that included a specific product (a flight on that route) during a time window.



Answer: Join chain: route (filter origin BOM → DEL) → flight (on route_id, filter scheduled_departure in July 2025) → booking (on flight_id) → pnr_group (on pnr_id) → payment (on pnr_id, WHERE payment_status = 'completed'). Then SUM(payment.amount). One important nuance: payment is at the PNR level, not per booking segment. If one PNR covers two flights and one is on BOM→DEL, the payment includes the other leg too. In a production system you would need fare-level pricing on each booking to split revenue accurately — this is a noted gap in the schema.



Why the examiner asks this: Reveals the payment aggregation nuance. Examiners often ask questions that expose known schema gaps to see if students are honest about limitations.

Q26

[Query Design]

How would you find all flights delayed beyond 2 hours?



Analogy: The operations dashboard is like a traffic monitoring system. You scan all incident records, filter only those where the delay duration exceeds your threshold, and map them back to the affected routes.



Answer: Query delay_record WHERE delay_minutes > 120, then join to flight (on flight_id) to get flight details. The index idx_flight_delay_tracking on (delay_minutes, flight_id) supports this range scan efficiently. A note: since delay_record is 1:M, a single flight may have multiple delay rows (ATC hold of 90 minutes + weather extension of 60 minutes). Depending on the requirement — either filter for any single event > 120 minutes, or SUM(delay_minutes) per flight — the query logic differs. Both are valid interpretations and the examiner may probe which one you mean.



Why the examiner asks this: Directly from the problem statement frequent queries list. The 1:M nuance on delay_record is a distinguishing detail.

SECTION 6 — ARCHITECTURE & HYPOTHETICALS

Q27

[Architecture]

What is CQRS and why did you mention it in the schema document for a DBMS assignment?



Analogy: In a large restaurant, the kitchen (write path) and the waiter's order history board (read path) are separate. Every new order goes to the kitchen (write). The board is updated in the background by a runner. A customer asking 'what did I order last week?' reads the board — the kitchen is not interrupted.



Answer: CQRS stands for Command Query Responsibility Segregation. It separates write operations (commands — INSERT, UPDATE, DELETE on the normalised MySQL database) from read operations (queries — served from a pre-computed read store like Redis or Elasticsearch). In our schema, BCNF normalisation is perfect for write integrity but creates deep multi-table joins for reads. Rather than denormalise the MySQL schema to speed up reads, we keep the source of truth clean and push pre-joined JSON documents to Redis. A PNR itinerary lookup becomes a single key-value fetch from Redis instead of a 6-table join on MySQL.



Why the examiner asks this: Shows you thought about real-world performance constraints beyond the DBMS assignment scope — exactly the kind of depth that impresses examiners.

Q28

[Architecture]

What is the role of the message queue (RabbitMQ/Kafka) in your architecture?



Analogy: When you order something on Amazon, the website immediately confirms your order and shows you a confirmation page. It does not wait for the warehouse to physically pick the item before responding. The warehouse notification is sent asynchronously in the background. The message queue is the relay system between the confirmation and the warehouse.



Answer: On a successful MySQL write (e.g. a booking confirmed), the application fires an event payload — BookingConfirmedEvent — onto the message bus without waiting for downstream processes. The API returns a response to the user immediately. Event consumers then pick up the payload asynchronously, execute the expensive multi-table join once, and push the result to the Redis read store. This means a seat availability cache update or a PNR itinerary cache invalidation never blocks the booking transaction itself. It also decouples the system so a Redis failure does not cause MySQL write failures.



Why the examiner asks this: Demonstrates awareness of asynchronous system design — relevant if the examiner pushes on performance under high booking traffic.

[Hypothetical]

Q29

Suppose the airline adds codeshare flights — where an Air India flight is also sold as a Vistara flight. What would you change in the schema?



Analogy: A codeshare flight is like a white-label product — the same physical flight operated by Air India but sold under Vistara's brand with a different flight number. The aircraft, crew, and gate are identical — only the marketing number and carrier code change.



Answer: We would need to: 1) Add an airline table (airline_id, airline_name, iata_carrier_code). 2) Add operating_carrier_id and marketing_carrier_id as FKs on the flight table — operating_carrier is who actually flies the plane, marketing_carrier is who sold it. 3) Add a codeshare_flight table or a codeshare_flight_number column on flight to store the partner carrier's flight number for the same segment. 4) booking.source_channel would expand to include the partner carrier's GDS booking channels. The core tables — passenger, booking, seat, payment — require no structural changes.



Why the examiner asks this: Codeshare is a well-known airline industry concept. The examiner is testing schema extensibility under a realistic business change.

[Hypothetical]

Q30

If you had to add a 'waitlist' feature — where passengers queue for a seat if the flight is full — what would you add to the schema?



Analogy: A waitlist is like a queue at a sold-out concert. When someone cancels, the first person in the queue gets offered the ticket automatically. You need to track who is in the queue and in what order.



Answer: We would add a waitlist table with: waitlist_id (PK), flight_id (FK), passenger_id (FK), fare_class_id (FK, the class they're waiting for), queue_position (INT, the priority order), requested_at (DATETIME), status (VARCHAR: 'waiting', 'offered', 'confirmed', 'expired'). booking_status already supports a 'waitlisted' status value. When a seat becomes available (availability_status changes from 'booked' to 'available'), the application queries waitlist WHERE flight_id = X AND status = 'waiting' ORDER BY queue_position ASC LIMIT 1, notifies that passenger, and sets their status to 'offered' with an expiry window.



Why the examiner asks this: Directly tests schema extensibility thinking — can you add a new feature without restructuring existing tables?

[Hypothetical]

Q31

If two passengers try to book the last available seat on a flight at exactly the same time, what happens? How does your schema prevent double-booking?



Analogy: Two people grab the same shopping cart item simultaneously in an online store. One of them must lose. The database needs a locking mechanism so that only one transaction can 'claim' the item before the other is allowed to proceed.



Answer: The seat table has an availability_status column with values ('available', 'held', 'booked'). In a production system, the booking flow would use a SELECT ... FOR UPDATE lock on the seat row within a transaction. Transaction 1 locks the row, changes status to 'held' (with hold_expires_at set), and commits. Transaction 2 attempts the same SELECT FOR UPDATE — it blocks until Transaction 1 commits, then reads the seat as 'held' and must pick another seat. The UNIQUE constraint on seat_id in the booking table adds a second safety net — even if two transactions race past the hold check, only one INSERT into booking with the same seat_id can succeed; the other fails with a unique key violation.



Why the examiner asks this: A classic concurrency control question. Shows you understand that schema design alone is not enough — transaction isolation matters.

[Hypothetical]

Q32

If a passenger cancels only their leg of a group booking, how does the schema handle that without affecting other passengers?



Analogy: Imagine four friends book a trip together under one invoice. One friend has to cancel. The invoice still stands for the other three — you just remove their line item and process a partial refund for their portion.



Answer: The schema handles this cleanly because booking is at the individual passenger-flight segment level. Cancelling one passenger's leg inserts a new row in booking_status (status = 'cancelled') for that specific booking_id — the other three passengers' booking rows are untouched. Their pnr_id is the same (shared PNR), but their booking records are independent. A partial refund is then created: a new row in refund linked to the original payment.payment_id with refund_amount = that passenger's fare share. The pnr_group itself is not cancelled — it remains active for the other three passengers.



Why the examiner asks this: Tests whether you understand the pnr_group → booking separation and the partial refund pattern.

[Hypothetical]

Q33

What would break in the schema if we removed the route table and put origin_airport_id and destination_airport_id directly on the flight table?



Analogy: If you wrote the full delivery address on every individual parcel for a courier company's 100 daily deliveries on the same route, and the postcode changed, you would need to update 100 parcels. Storing the route once and referencing it is cleaner and avoids that update anomaly.



Answer: Three problems: 1) BCNF violation — distance_km is a fact about the corridor (origin → destination pair), not about a specific flight instance. If stored on flight, it would be repeated across every daily flight on that route. 2) Update anomaly — if distance_km is recalculated (e.g. a more accurate geodesic measurement), every individual flight row must be updated. 3) Route-level analytics break — the query 'aircraft utilisation across different routes' requires grouping by route, which is impossible without a stable route_id. The index idx_flight_route_date also relies on route_id as the first column.



Why the examiner asks this: *Tests the ability to articulate what a table provides — and what you lose by removing it.*